

Balancing a Binary Search Tree

Design and Analysis of Algorithms

Brian C. Dean

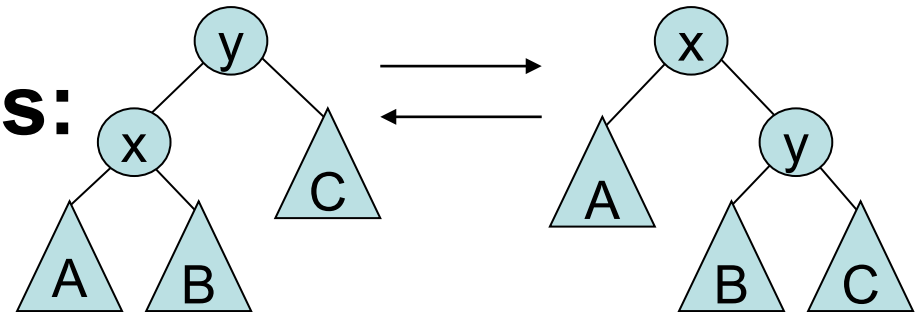
**School of Computing
Clemson University**



Balancing a BST

- There are many ways to modify the insert and delete operations of a BST to keep it balanced:
 - **Worst-case** mechanisms: $h = O(\log n)$ always.
AVL trees, $BB[\alpha]$ trees, red-black trees
 - **Amortized** mechanisms: starting with an empty tree, any sequence of k operations takes $O(k \log n)$ time.
splay trees, “batch” rebalancing methods
 - **Randomized** mechanisms: always balanced “with high probability”.
randomly-balanced BSTs, treaps

- Many of these are based on **rotations**:

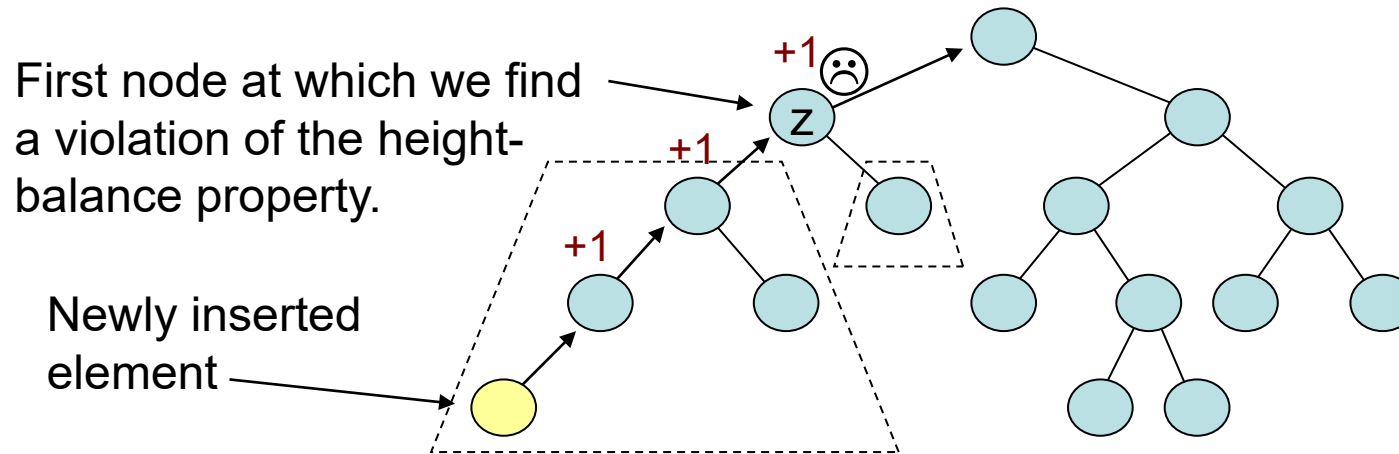


Example of Balancing with Rotations: AVL Trees

- AVL: Adel'son-Vel'skiĭ, and Landis '62.
- A binary tree is **height balanced** if for every node, its left and right subtrees differ in height by at most 1.
- Easy to show by induction that height-balanced trees are balanced.
- After each insertion or deletion, we restore the height balance property in $O(\log n)$ time by performing carefully chosen rotations.
- **Claim:** A height-balanced tree of height h contains $\geq F_h$ nodes, where F_h is the h^{th} Fibonacci number.
- We know $F_h = \Theta(\Phi^h)$, where $\Phi = (1+\sqrt{5}) / 2 \approx 1.618$. Therefore, an n -element height-balanced tree can have height at most $O(\log n)$.

Restoring Balance After an Insertion

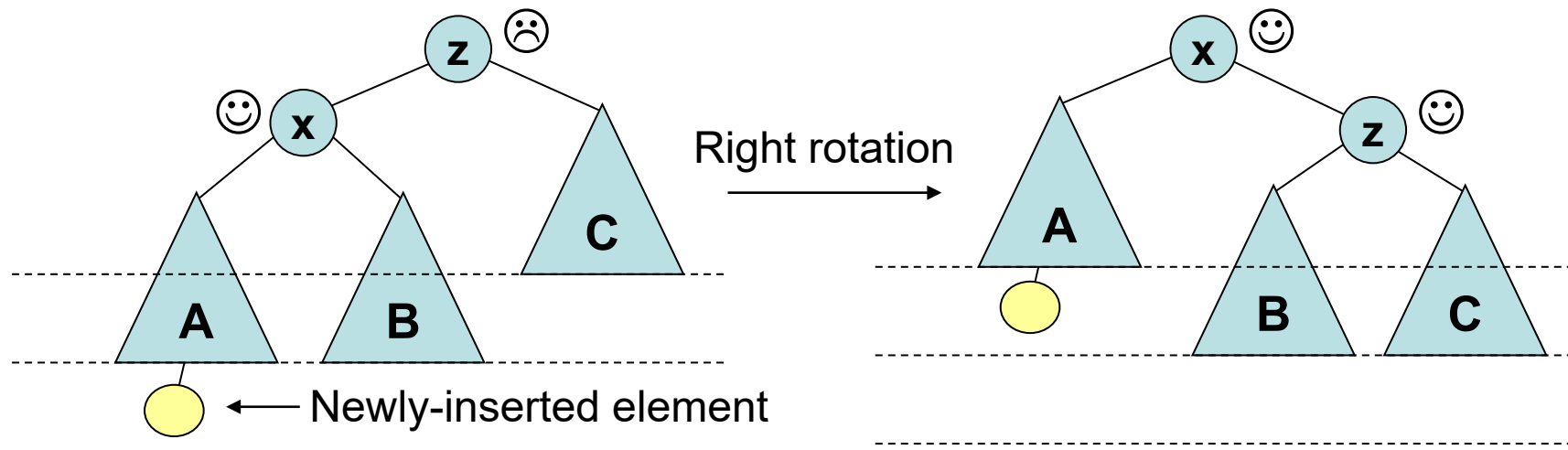
- After an *insert*, walk back up the insertion path and increment augmented subtree heights.
- Stop at the first node (z in the figure below) at which we encounter a height-balancing violation (a height imbalance of exactly 2).



- Using 1 or 2 rotations, we will be able to rebalance our tree at z.
- The height of z's subtree will decrease by 1 to its original value, so we don't need to visit the remaining nodes on the insertion path above z.

Restoring Balance After an Insertion

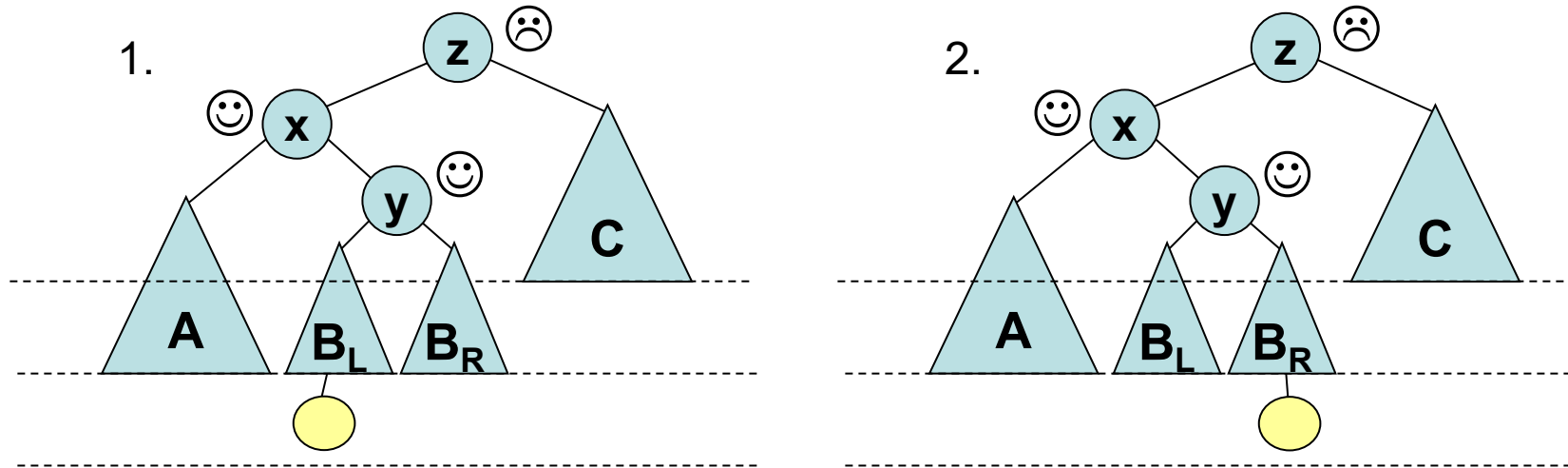
- Let's suppose we have an imbalance at node z due to an insertion that makes z's left subtree too tall (the same argument will apply to the right subtree as well...)
- Easy case:** Insertion into subtree A makes it too tall:



- Harder case:** Insertion into subtree B makes it too tall...

Restoring Balance After an Insertion

- **Harder case:** Subtree B too tall. Two possible subcases:



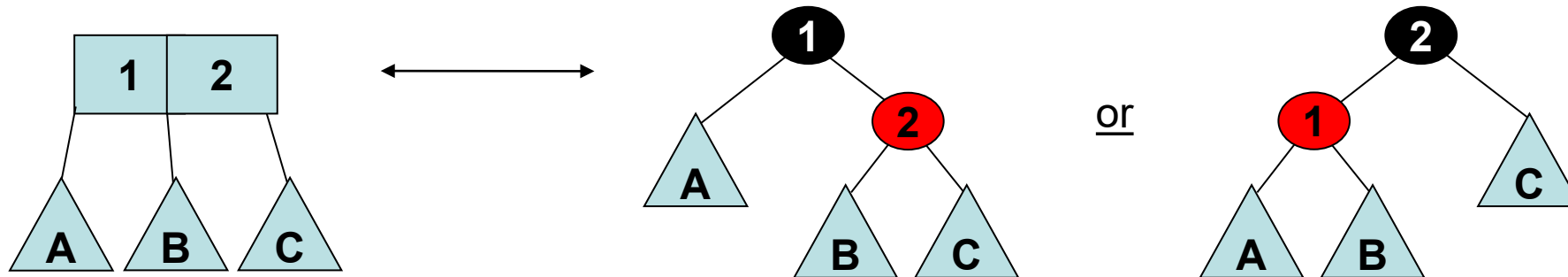
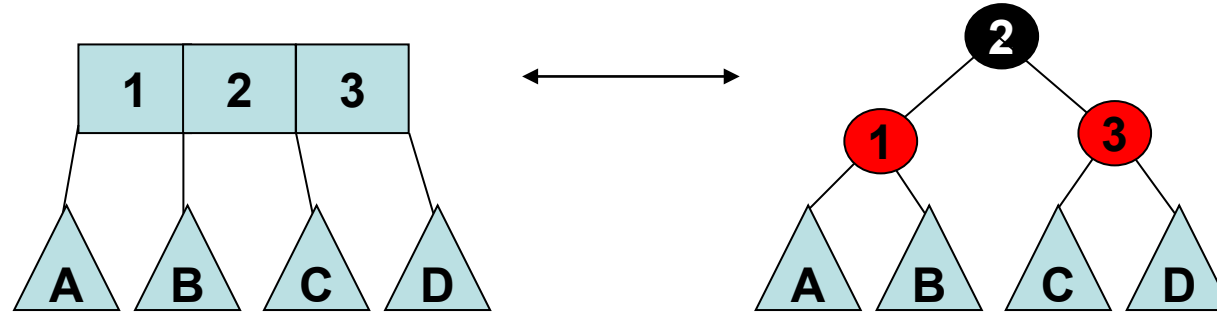
- In both cases, a left rotation about the edge (x, y) brings us back to the previous case (with only A being too tall). So 2 total rotations needed for these cases.

Red-Black Trees

- Popular in practice, despite cumbersome details needed for implementation.
- **Red-Black Property:** Each node colored red or black, so that:
 - No two red nodes can be adjacent (parent-child).
 - Same # of black nodes on every root-leaf path.
- One can show by induction that this property guarantees balance.
- After any insert or delete, one can restore the property if it breaks by using $O(\log n)$ carefully-chosen rotations and/or node recolorings.

Further Evidence we Don't Need to Study Red-Black Trees...

- Interesting bit of trivia: a “2-3-4” tree (a B-tree with $B=2$) is “equivalent” to a red-black tree!



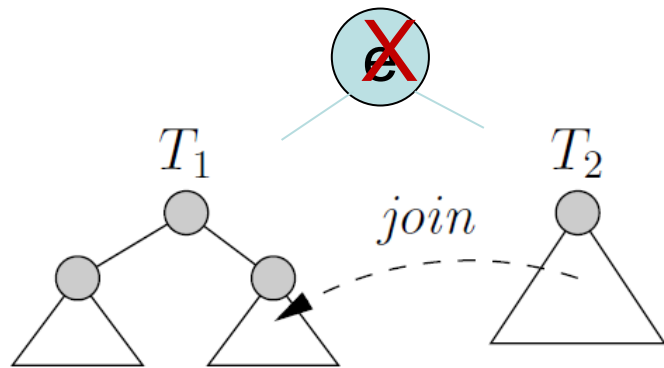
Recall: Randomly-Built BSTs

- Build a BST on n elements by inserting in random order $\rightarrow$ the tree balanced with high probability.
- However, the user may not be so kind as to insert elements in random order...
- We can address this by doing some carefully random modifications during each insert and delete so the tree is always in a state that is “as if it was just randomly built from scratch”.
 - Let’s call this the **Randomized BST (RBST)** property.
 - Useful characterization: within each subtree, each element is equally likely to be at the root.

Easy to Maintain RBST Property, so Height Always $O(\log n)$ w.h.p.!

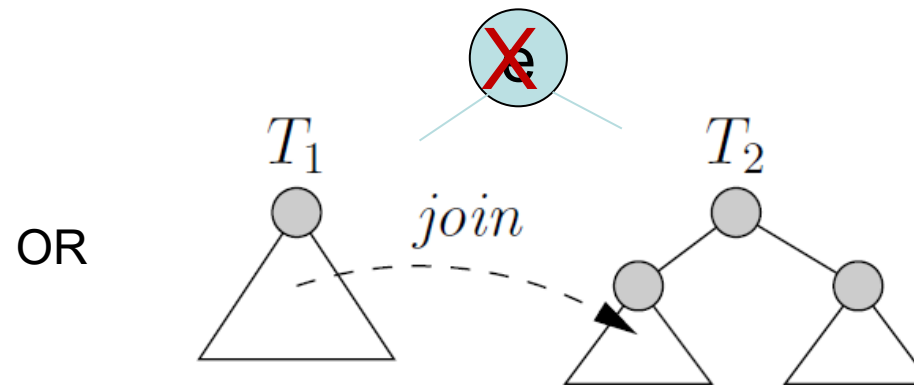
- To insert an element e into an $(n - 1)$ -element tree:
 - With probability $1/n$, insert e at the root (e.g., with split)
 - Otherwise (with probability $1 - 1/n$), recursively use same method to insert into the left or right subtree of the root
- To delete e , replace e with the join of its left and right subtrees:

With probability proportional to $\text{size}(T_1)$:



$T1 \rightarrow \text{right} = \text{join}(T1 \rightarrow \text{right}, T2);$

With probability proportional to $\text{size}(T_2)$:



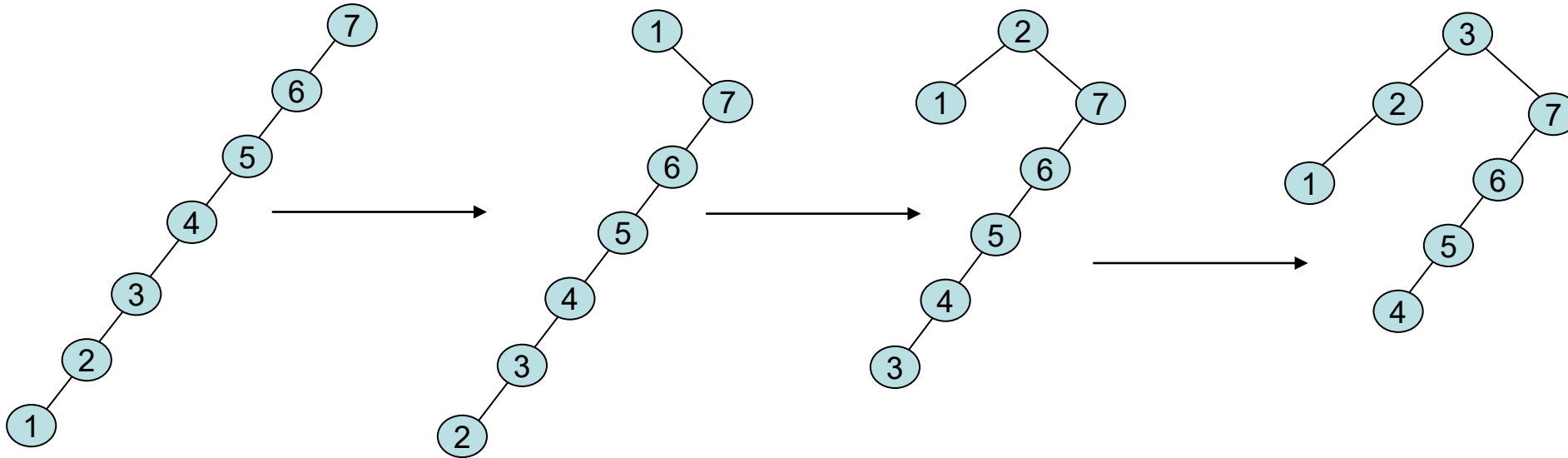
$T2 \rightarrow \text{left} = \text{join}(T1, T2 \rightarrow \text{left});$

Amortized Balancing: Splay Trees

- Surprisingly simple way to achieve the same logarithmic performance as a balanced BST, even though strict balance may not always hold...

Rotating Elements to the Root

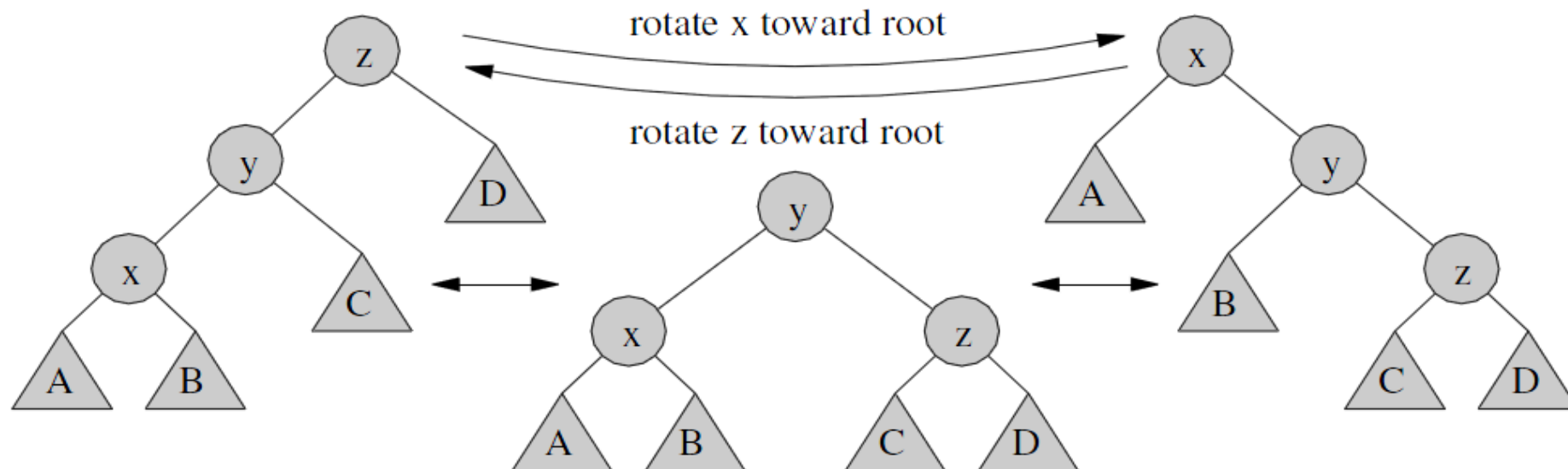
- **Idea:** every time an element is accessed, rotate it all the way up to the root.
- Unfortunately, this doesn't always help.



- Example: what if we access 1, 2, 3, 4, 5, 6, 7, 1, 2, 3, 4, 5, 6, 7, ...?

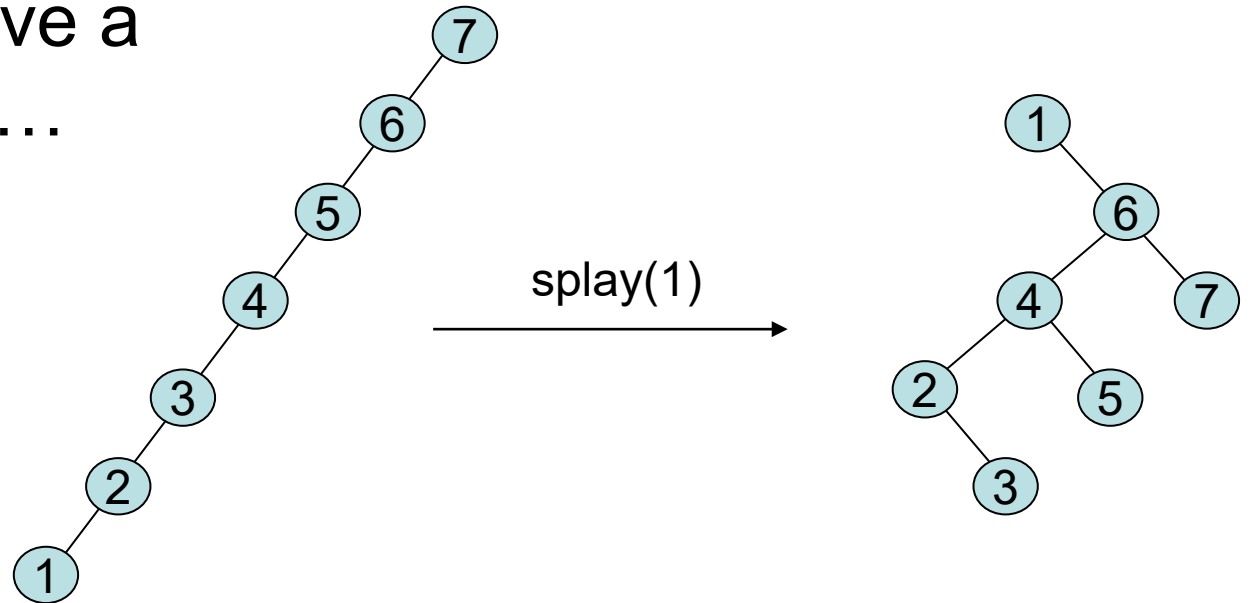
Rotate to Root : An Improvement?

- **New idea:** When an element is accessed, “splay” it:
 - Rotate up to root as before.
 - If at any point element is “in-line” with parent and grandparent, do next two rotations out of order.



Rotate to Root : An Improvement?

- **New idea:** When an element is accessed, “splay” it:
 - Rotate up to root as before.
 - If at any point element is “in-line” with parent and grandparent, do next two rotations out of order.
- Anecdotally, this seems to leave a “flatter” tree shape in its wake...

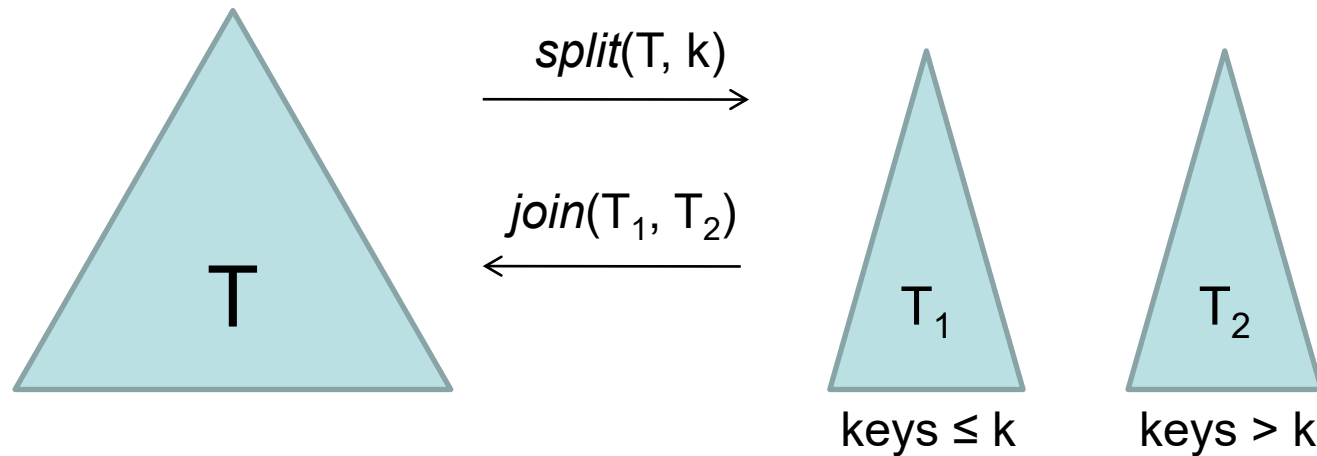


Splay Trees

- Normal BST that splays elements when accessed:
 - *find*(*e*) : Find as *e* usual, then *splay*(*e*).
 - *insert*(*e*) : Insert as *e* usual, then *splay*(*e*).
 - *delete*(*e*) : *splay*(*e*), then replace *e* with join of children
- Called a **self-adjusting** tree, since it “blindly” modifies its structure according to simple local update rules.
- No augmented information (e.g., subtree sizes) needed!
- Any sequence of *k* operations (starting from an empty tree) takes $O(k \log n)$ time $\rightarrow O(\log n)$ amortized time / operation!
- Somewhat “magical” (i.e., we don’t have great insight why...)

Split and Join Easy on a Splay Tree!

- $split(T, k)$: Splay k to root (or $pred(k)$ if k not there), then remove right subtree.
- $join(T_1, T_2)$: Splay the maximum in T_1 to the root and attach T_2 as right subtree (or vice versa)

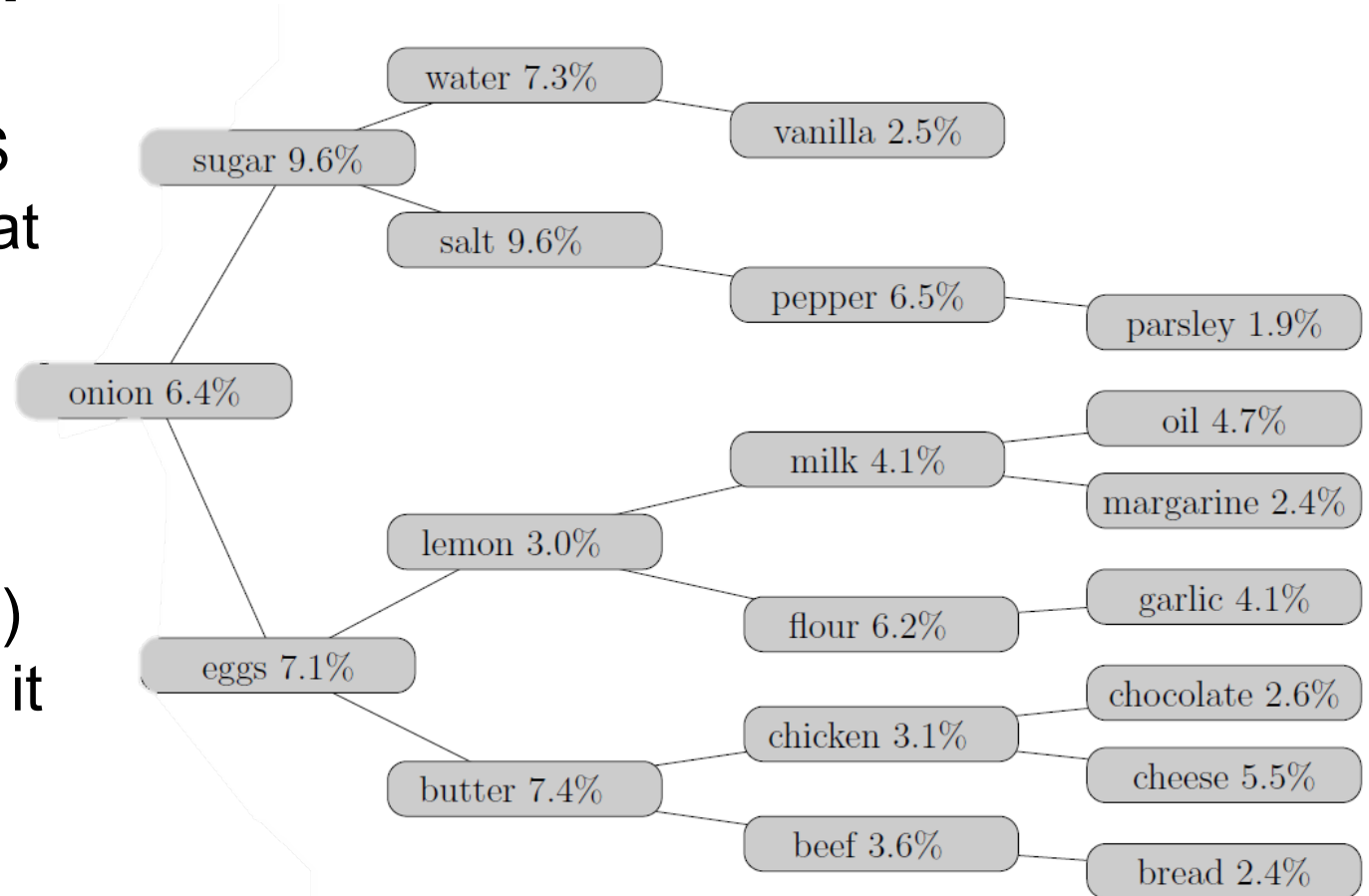


Splay Trees : Remarkable Properties

- **Static Optimality Theorem:**

Build a static “optimal” BST, T .
Given some access sequence S
(which accesses each element at least once). Let X be the total amount of time T spends processing S .

A splay tree will spend only $O(X)$ time processing S , even though it doesn't know the access sequence or frequencies in advance!



Splay Trees : Remarkable Properties

- **Dynamic Optimality Conjecture:**

For any access sequence S , a splay tree spends only $O(X)$ time, where X is the best time we could have achieved with *any* dynamic BST, even one that knows the access sequence in advance and performs rotations over time to optimize its structure accordingly.